

Install and Usage

What you are installing

Three folders of Markdown and static assets. There is no build step, no package manifest, no service and nothing to compile. The installers only copy files.

```
security-audit/           the audit engine
security-audit-compare/   the comparison engine
security-code-review/     the orchestrator you actually invoke
```

Keep them as siblings — `security-audit-compare` reads `security-audit`'s cost collectors by relative path.

Requirements

Requirement	Notes
Claude Code	Provides the <code>Grep</code> / <code>Glob</code> / <code>Read</code> tools the scan runs on. Ripgrep is bundled
A shell	bash or PowerShell — you already have one. Used to assemble the report
Cost accounting	Uses whichever of Python / bash+awk / PowerShell 5.1 is present. All three produce identical numbers. Without any, the audit still runs and cost shows "not collected"
Network / MCP	Not required. Only improves dependency-advisory checking

Nothing else. No pip, no npm, no Docker, no scanner binaries, no interpreter to fetch. If anything ever asks you to install something to run an audit, that is a bug.

1. Install

Windows (PowerShell)

```
cd "path\to\Security code review skills"

# Every project on this machine -> ~\.claude\skills
.\install.ps1

# One project only -> <project>\.claude\skills
.\install.ps1 -Scope project -Target C:\src\acme

# Overwrite an existing install without prompting
.\install.ps1 -Force
```

If PowerShell blocks the script:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\install.ps1
```

macOS / Linux / Git Bash

```
cd "path/to/Security code review skills"

./install.sh                # ~/.claude/skills
./install.sh --project /src/acme # /src/acme/.claude/skills
./install.sh --force
```

By hand

The installers do nothing you cannot do yourself:

```
cp -r security-audit security-audit-compare security-code-review ~/.claude/skills/
```

```
Copy-Item -Recurse security-audit,security-audit-compare,security-code-review
"$HOME\.claude\skills\"
```

Verify

Start a **new** Claude Code session (skills are discovered at startup), then:

```
/help
```

The skill list should include `security-code-review`, `security-audit` and `security-audit-compare`. If it does not, check that each folder contains a `SKILL.md` directly inside it — `~/.claude/skills/security-audit/SKILL.md`, not `~/.claude/skills/security-audit/security-audit/SKILL.md`.

Where to install

Scope	Path	Use when
User	<code>~/.claude/skills/</code>	You review several projects. Available everywhere
Project	<code><project>/~/.claude/skills/</code>	The team should get it by cloning. Commit it

Project scope is the right default for a modernization engagement: commit the three folders and everyone on the team gets the same audit rules, the same scoring weights and the same report format.

2. Run a review

Interactive — VS Code or the terminal

```
/security-code-review
```

It asks for three things:

Before I start, three things:

1. Project name? (used for the report folder and titles)
2. Legacy source path? (leave blank or say "none" if there is no legacy codebase)
3. Modernized source path? (required)

Then it validates the paths, shows you the plan, and waits:

```
Project:      AcmeBilling
Legacy:       C:\src\acme\legacy      (2,140 files)  -> audit, no remediation
Modernized:   C:\src\acme\modern      (148 files)   -> audit + remediation
Depth:       standard
Output:       C:\reports\AcmeBilling\
              3 reports: Legacy, Modernized, Comparison

Proceed?
```

Check the file counts. A number two orders of magnitude off the expected is the cheapest possible signal that a path points somewhere unintended — and this is the last moment it is free to fix.

On approval it runs the phases in order and announces when it is done.

Skipping the questions

Anything on the command line is not asked for again:

```
/security-code-review --name "AcmeBilling" --legacy C:\src\acme\legacy --modernized
C:\src\acme\modern
```

Modernized only, no legacy codebase

Leave the legacy answer blank, or:

```
/security-code-review --name "AcmeBilling" --legacy none --modernized C:\src\acme\modern
```

Produces the Modernized report only. There is no Comparison report, because there is nothing to compare against — it says so rather than inventing a baseline.

Non-interactive — UI, CI, or a script

Supply everything plus `--yes`:

```
claude -p '/security-code-review \
--name "AcmeBilling" \
--legacy /src/acme/legacy \
--modernized /src/acme/modern \
--out /reports \
--depth standard \
--yes'
```

```
claude -p '/security-code-review --name "AcmeBilling" --legacy C:\src\acme\legacy --modernized
C:\src\acme\modern --out C:\reports --yes'
```

There is no separate headless code path — the same skill serves both, which is why the two cannot drift apart.

3. What you get

```
C:\reports\AcmeBilling\  
├─ AcmeBilling - Legacy - Security analysis report. - 2026-09-11.html  
├─ AcmeBilling - Modernized - Security analysis report. - 2026-09-11.html  
├─ AcmeBilling - Comparison - Security analysis report. - 2026-09-11.html  
└─ .security-audit\  
    ├── run.json          this run's manifest: paths, depth, per-phase status  
    ├── legacy\          findings.json, inventory.json, raw-hits.json, cost.json  
    ├── modernized\  
    │   (same)  
    └─ comparison\  
        comparison.json, functional-areas.json, pairings.json, cost.json
```

Double-click any report. No server, no build, no internet.

Start with the Comparison report if you have all three — it carries the verdict, the bucket counts, and the total cost of the whole review.

Do not delete `.security-audit/`

It is not a temp directory. It holds:

- `findings.json` — the machine-readable record, and the input to any future comparison or `--baseline` diff;
- the per-phase cost records;
- `run.json`, which lets a failed run resume without repeating what was already paid for.

Deleting it means paying for the audit twice. If you circulate the folder, the three HTML files are self-contained — send those.

4. Options

Depth

```
/security-code-review --depth quick      # ~1x  CI gate, re-run after fixes  
/security-code-review --depth standard  # ~3x  default; release review  
/security-code-review --depth deep      # ~8x  pre-production, regulated, first audit
```

Both sides always use the same depth. A `deep` legacy audit compared against a `quick` modernized one produces bucket counts that measure the effort spent rather than the codebases.

Output location

```
/security-code-review --out C:\reports
```

The project folder is created **inside** `--out` . Default is the current working directory.

5. Running the pieces separately

Audit one codebase

```
/security-audit C:\src\acme\modern --name "AcmeBilling"
```

A normal standalone audit: findings, score, full remediation, roadmap, cost. No migration context, no variant label in the filename.

```
/security-audit C:\src\acme\legacy --name "AcmeBilling" --mode analyze --variant legacy
```

Diagnosis only — no remediation, and the report says the omission was deliberate.

Compare two audits you already have

```
/security-audit-compare \  
  --legacy      C:\reports\AcmeBilling\.security-audit\legacy\findings.json \  
  --modernized C:\reports\AcmeBilling\.security-audit\modernized\findings.json \  
  --name "AcmeBilling" --out C:\reports\AcmeBilling
```

Useful when the two audits were run days apart, or when you re-audited one side after fixes.

Re-audit after fixing things

```
/security-audit C:\src\acme\modern --name "AcmeBilling" \  
  --baseline C:\reports\AcmeBilling\.security-audit\modernized\findings.json
```

Marks each finding **New**, **Persisting** or **Resolved** against the previous run, so you can show progress rather than re-reading the whole report.

6. Cost

A review costs real money — the worked example in `security-audit-compare/examples/` cost about **\$6.24** across all three phases for a 148-file modernized tree and a 2,140-file legacy tree at `standard` depth.

Rough guidance:

Factor	Effect
<code>--depth quick</code> → <code>standard</code> → <code>deep</code>	roughly 1× → 3× → 8×
Codebase size	more files means more triage, the expensive part
Legacy phase	adds roughly the cost of a second audit
Comparison phase	typically the cheapest of the three

Every report states its own cost, and the Comparison report states the total for the run. Figures use published list rates and exclude enterprise, Batch API and partner-platform discounts.

To skip cost accounting entirely, pass `--no-cost` to the individual skills.

7. Troubleshooting

The skill does not appear in `/help`

Skills are discovered at session start — start a new session. Then check the layout:

```
~/ .claude/skills/security-audit/SKILL.md          correct
~/ .claude/skills/security-audit/security-audit/  one folder too deep
```

"Report could not be rendered" in the browser

The embedded JSON did not parse; the page names the error. The analysis is not lost — it is in `.security-audit/<phase>/findings.json`. Fix the JSON and re-concatenate:

```
cat security-audit/assets/report.part1.html \
    .security-audit/modernized/findings.json \
    security-audit/assets/report.part2.html > "report.html"
```

```
Get-Content ".\security-audit\assets\report.part1.html", `
    ".\security-audit\modernized\findings.json", `
    ".\security-audit\assets\report.part2.html" -Raw |
Set-Content "report.html" -Encoding utf8
```

Cost shows "not collected"

No cost collector could run, or the transcript was unreadable. The audit is unaffected — only the cost section. To check whether Python is visible to the skill:

```
bash security-audit/bin/ensure_python.sh --quiet
```

```
powershell -NoProfile -File .\security-audit\bin\ensure_python.ps1 -Quiet
```

It prints a path if it found one. Nothing printed means it will fall back to bash+awk or PowerShell, both of which produce identical numbers — so this is rarely worth chasing.

On Windows, note that `%LOCALAPPDATA%\Microsoft\WindowsApps\python.exe` is a 0-byte App Execution Alias that resolves on PATH but only opens the Microsoft Store. The resolver probes candidates by *executing* them rather than trusting `command -v`, specifically to avoid picking it.

It asks to download Python

It should only ever do that with `--allow-download`, which requires your explicit yes. Decline it — the audit works without Python, and the bash and PowerShell collectors give the same numbers. If it asked without being passed that flag, that is a bug.

"Both paths resolve to the same directory"

Exactly what it says. Comparing a tree with itself produces an all-`inherited` report and a zero delta. Check which path is wrong.

The comparison was skipped

Three possible reasons, and the run tells you which:

Reason	Fix
No legacy path was given	Expected. There is nothing to compare
One audit failed	Check <code>run.json</code> ; re-run that phase
<code>security-audit-compare</code> is not installed	Install it alongside the other two

Scores differ between the audits and the comparison

They should not — the comparison never recomputes a score, it copies both. If they differ, the two audits used different scoring profiles. Re-run one of them with the same profile; do not rescale.

8. Embedding in another project

Copy the three folders into that project's `.claude/skills/` and commit them. Nothing else is needed — no configuration file, no registration step, no environment variable.

For a UI integration, the contract is:

Input	<code>--name</code> , <code>--legacy</code> , <code>--modernized</code> , <code>--out</code> , <code>--depth</code> , <code>--yes</code>
Progress	The orchestrator's phase announcements on stdout
Result	<code>run.json</code> for machine-readable status and paths
Display	The three HTML files — static and self-contained: serve, iframe, or attach unchanged

Because the reports have no external dependencies, they can be served from any static host, stored as build artifacts, or emailed without anything breaking.

9. Maintaining the rule packs

Only relevant if you are editing `security-audit/rules/*.json`. After any change:

```
python security-audit/tests/run_corpus.py
```

It verifies that every pattern compiles under ripgrep's Rust regex engine (no lookahead, no backreferences) and that rules fire on the vulnerable fixtures **and stay silent on the safe ones**.

The silence half is the one that matters. A rule that flags parameterized SQL teaches engineers to ignore the report, and that loss is permanent. Every new rule should arrive with at least one `reject` case drawn from its own `falsePositiveHints`.

This is the only part of the system that needs Python, and only for maintainers — never for users.